A Minimal Verifiable Profile and Validator Path for Single-Operation Accountability in FDO and Data-Space Settings

Zhang Bin
Independent Research Scholar
China
joy7759@gmail.com

Abstract—Agent-mediated data operations in FDO and data-space settings are often visible through logs, yet logs alone do not give another party a compact, independently checkable account of one operation. They preserve runtime history, but they do not automatically package actor, subject, policy, evidence, and validation relations as one reviewer-facing object for later independent external review. This paper presents a minimal verifiable profile for single-operation accountability and a validator path that exercises that profile on a bounded artifact surface. The current implementation includes a profile specification, a JSON Schema, a reference validator CLI, one valid example, three controlled invalid examples, test anchors, and one runnable walkthrough. The validator checks four properties: schema conformance, reference closure, cross-field consistency, and bounded integrity recomputation. Together, the schema, validator, examples, tests, and walkthrough form a small inspectable closure for one repo-backed operation context. The contribution is intentionally narrow: it shows that one agent-mediated operation in an FDO or data-space setting can be expressed and checked as a single-operation accountability object.

Index Terms—execution evidence, operation accountability, data spaces, FAIR digital objects, validation, provenance, artifact verification

I. Introduction

FDO and data-space settings increasingly rely on object-mediated operations carried out by software agents. An agent may read an object, update metadata, emit derived references, and do so under explicit policy conditions. Later review, however, is often weaker than execution itself. A runtime may expose traces and logs, yet an external reviewer still has to reconstruct what was acted on, under which rule surface, and whether the resulting record is internally coherent. The digital-object lineage explains why this matters: object identity and machine-handled interaction are central at the systems level, but they do not by themselves supply a compact accountability object for one operation [1]–[3].

That gap is narrower than observability. A trace is useful for debugging, but debugging and later accountability are not the same review task. A reviewer-facing record has to state the actor, subject, operation, policy surface, evidence links, and validation state for one operation in a form that can be checked directly. It also has to make those relations explicit enough that a third party can test them without reconstructing the full runtime. When those relations remain implicit or scattered across runtime records, later review depends on reconstruction rather than inspection.

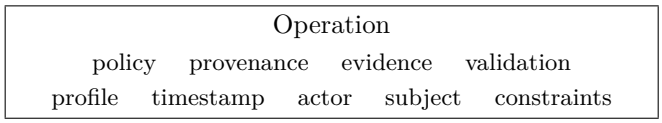

Fig. 1. Operation-centered structure of the accountability object.

This paper takes the smallest path we can defend. It treats execution evidence as a single-operation accountability object with an explicit profile, not as a larger telemetry stack. The shipped artifact couples that profile to a JSON Schema and a validator that checks four properties: schema conformance, reference closure, cross-field consistency, and bounded integrity recomputation. The point is not richer logging. The point is a small object that another party can load, validate, and interpret without replaying a larger runtime. In that sense, the paper is closer to provenance validity work than to larger interoperability proposals [4], [5].

The current repo-backed surface is deliberately tight: one valid example, three controlled invalid examples, tests, a command-line validation path, and one runnable walkthrough. That boundary is also the paper's scope boundary. The paper claims that this one context can be represented and checked as a bounded accountability object. It does not claim more than the artifact shows.

Within that boundary, the contribution is practical. First, the paper defines a minimal operation-centered profile with explicit top-level relations for actor, subject, operation, policy, constraints, provenance, evidence, and validation. Second, it provides a validator path that turns the profile from a descriptive format into an executable check surface. Third, it documents a small artifact closure in which one positive case, three negative cases, tests, and one walkthrough can be inspected together. The result is not simply a data format. It is a small reviewable package in which the object model, validation rules, and exercise path stay aligned.

II. Method

The method is an operation-centered object model plus a bounded validation path. The object model is encoded as one JSON record whose top-level sections are profile, statement_id, timestamp, actor, subject, operation, policy, constraints, provenance, evidence, and validation. These sections are not independent metadata buckets. They form one relation layer around a single operation statement, and that shared relation layer is what later validation works on.

Each section has a bounded role. profile fixes identity and version. statement_id and timestamp identify one statement instance. actor records who performed the operation. subject records the primary object acted on. operation records the action and its input and output references. policy and constraints record the governing rule surface. provenance ties actor, subject, operation, and input or output references together. evidence records emitted references, artifacts, and integrity fields. validation records how the statement is to be checked.

The design keeps the operation at the center. The operation links outward to subject, inputs, outputs, and policy. Provenance repeats those references so that they can be checked rather than inferred. Evidence binds the statement to emitted references and artifacts. Validation binds the statement to the objects and checks used to review it later. This arrangement supports one bounded question: whether a later reader can inspect one statement and determine that its internal relation layer is coherent.

The shipped JSON Schema provides the structural contract for this object. At that layer, the method enforces required fields, fixed profile identifiers, value types, formats, and bounded enums. Schema checks are therefore the first gate. A malformed instance does not enter later stages.

The reference validator then applies four stages. Stage one checks schema conformance. Stage two checks reference closure. Stage three checks cross-field consistency. Stage four performs bounded integrity recomputation. Taken together, these four stages give the artifact a stable failure grammar: malformed structure, broken references, inconsistent relations, or mismatched integrity fields.

The current implementation exposes the same path through a reference CLI. The CLI reads one profile instance and returns a machine-readable JSON report. On success, the report indicates a clean pass with no issues. On failure, it records stage-specific issues and exits nonzero.

The shipped examples, tests, and runnable walkthrough all exercise this same path. One valid case serves as the positive witness. Three invalid cases each break one primary rule class: required-field completeness, reference closure, or the evidence-to-policy relation. The walkthrough shows that the same validation path can be rerun on a single end-to-end case rather than only on static files.

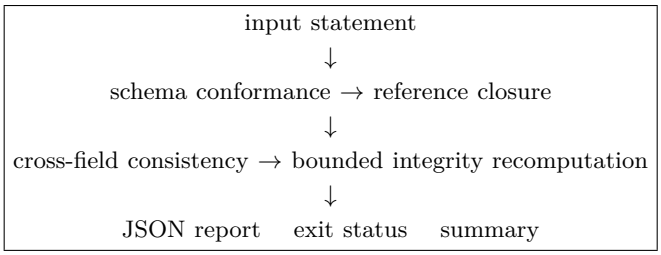

Fig. 2. Four-stage validation path for one profile instance.

III. Evaluation

This evaluation asks a narrow question: does the shipped artifact expose a bounded and diagnosable validation surface for one repo-backed context? The evaluated surface includes the profile specification, JSON Schema, reference validator CLI, one valid example, three invalid examples, tests, and one runnable walkthrough.

The valid example serves as the positive witness. Its expected outcome is a passing report with no issues. The three invalid cases are controlled negative witnesses. They cover a missing required field, an unresolved reference, and a broken policy relation. The point of this negative set is not completeness. The point is to show that the validator distinguishes several failure classes inside the current profile boundary and reports them in a machine-readable form.

Tests and the CLI anchor this boundary. The tests check the operation surface and the command-line exposure. The CLI returns a JSON report and a non-zero exit on failure, so the same path can be exercised by a reviewer rather than only by internal code. The runnable walkthrough extends that path one step further by showing one end-to-end case. It emits a statement, validates it, and produces review artifacts.

What this evaluation establishes. It establishes that the current artifact supports one passing path, three controlled failing paths, one runnable walkthrough, and one reusable validation interface. It also establishes that the artifact boundary is inspectable: the schema, validator, examples, tests, and walkthrough are aligned around the same object model under one shared validation interface.

What this evaluation does not establish. It does not establish behavior beyond the current repo-backed context. It does not show a broader scenario family, and it does not by itself justify stronger generality claims. That narrower conclusion is intentional: the evaluation is meant to show closure, not scope.

IV. Related Work

Digital-object work provides the overall context. The digital object lineage emphasizes persistent identity and machine-handled interaction, while DOIP makes that interaction explicit at protocol level [1], [2]. Recent FDO work studies those ideas at a larger systems scale than the present paper [3]. That larger lineage matters here because

the present paper borrows the object-centered setting but not the larger scope.

Provenance work is the closest conceptual neighbor. PROV-DM defines general models for entities, activities, and agents, and PROV-Constraints makes explicit that provenance representations can carry validity conditions [4]–[6]. The present paper shares the emphasis on explicit relations and checkable consistency, but narrows the target to one operation-centered object.

Logs and traces remain essential for observability and debugging. The present paper does not compete with telemetry systems. It isolates a different review target: a compact artifact that a third party can check after one operation rather than a larger runtime transcript.

Artifact-oriented verification offers another useful comparison point. in-toto shows how explicit step records can support downstream verification, and ACM artifact-review guidance emphasizes inspectable, exercisable research objects [7], [8]. The present paper adopts that smaller artifact posture for one operation accountability object and keeps the contribution reviewable at workshop scale.

V. Limits and Conclusion

The current repo-backed boundary is small. It contains one valid context, three controlled invalid cases, one validator path, tests, and one runnable walkthrough. That boundary is a limitation, but it is also the design choice that makes the artifact readable and checkable.

Minimality here is methodological rather than accidental. By keeping the operation at the center and by restricting validation to four explicit checks, the paper avoids turning one checkable claim into several uncheckable ones. The result is a paper that can state exactly what the artifact supports and stop there.

The conclusion is therefore narrow. One agent-mediated operation in an FDO or data-space setting can be represented as a single-operation accountability object with a concrete validation path. The profile, schema, validator, examples, tests, and walkthrough form a small inspectable closure that reviewers can understand without reconstructing a much larger runtime. Future extensions can broaden contexts only after the same closure exists for them.

References

[1] R. Kahn and R. Wilensky, "A framework for distributed digital object services," International Journal on Digital Libraries, vol. 6, no. 2, pp. 115–123, 2006.

[2] DONA Foundation, "Digital object interface protocol specification, version 2.0," DONA Foundation, Protocol Specification, Nov. 2018.

[3] S. Soiland-Reyes, C. Goble, and P. Groth, "Evaluating FAIR digital object and linked data as distributed object systems," PeerJ Computer Science, vol. 10, p. e1781, 2024.

[4] L. Moreau and P. Missier, "PROV-DM: The PROV data model," World Wide Web Consortium, W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-dm/

[5] J. Cheney, P. Missier, and L. Moreau, "Constraints of the PROV data model," World Wide Web Consortium, W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-constraints/

[6] P. Groth and L. Moreau, "PROV-overview: An overview of the PROV family of documents," World Wide Web Consortium, W3C Working Group Note, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-overview/

[7] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in 28th USENIX Security Symposium (USENIX Security 19). USENIX Association, 2019, pp. 1393–1410. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[8] Association for Computing Machinery. (2020) Artifact review and badging - current. Version 1.1. [Online]. Available: https://www.acm.org/publications/policies/artifact-review-and-badging-current